ePolicies AI Assistant – What Changed and How It Affects the Project

This document lists every change made after testing the assistant on the laptop with the real 235 policies: what problem was seen, what was changed, and what it means for answers, speed and cost. All changes are on GitHub (https://github.com/charan4590/epolicies-rag). Backend tests: **149 passed**; website type and lint checks pass.

1. Summary

#	Problem seen during testing	Change	Effect
1	"How many policies are there?" → *insufficient evidence*	Collection questions answered from the document index	Instant, exact answer; no Gemini call
2	Misspelt follow-up got the "Hello! I'm the ePolicies AI Assistant" reply	Only greetings/thanks skip the search	Every real question is searched
3	Misspelt words ("jpoint partership") broke keyword search	Words found in no policy → Gemini corrects the question first	Better results for typos; +1 Gemini call for those questions
4	"abortion" and "termination of pregnancy" gave different answers	Synonyms and acronyms added to keyword search	Different wordings find the same passages
5	Answer said "refer to the … policy" but that policy was not read	Referenced policies are searched in the next round	More complete answers; fewer yellow banners
6	Unclear why a lower-RRF passage was first	Diagnostics explain the order and mark relevant passages (✓)	Ranking is transparent
7	Yellow warning on answers whose citations were all verified	Quiet grey "Partial answer" note instead	Less alarming, still honest
8	Broad questions were incomplete	More passages per answer (12–16 instead of 8–12)	More complete answers; a few more tokens
9	Weekly update needed many manual commands	`update-windows.bat`	One double-click with a Y/N check

Kept as before (on request): RRF counts every rank (`RRF_K=60`), and `GEMINI_TEMPERATURE` stays at **0.1** so answers can vary in wording.

2. Each change in detail

2.1 Questions about the collection ("how many / list all policies")

- **Problem:** the assistant answers from the 8–16 passages that best match the question. No passage says "there are 235 policies", so it replied *Insufficient evidence – Not found: the total number of policies*.
- **Change:** questions such as "How many policies are there?", "Name all the policies", "List all policies about Medicare" are recognised and answered from the list of indexed documents in PostgreSQL.

    - New file `backend/app/agents/catalog.py`; `repository.indexed_document_titles()`; `rag_service._catalog_answer()`.
    - Questions about policy *content* ("How many policies cover abortion?") still go through the normal search.
    - Documents with the same title are counted separately and shown with their file name.
- **Effect:** correct count and full list, instantly, with **no Gemini call** (no cost). The answer shows the note "Answered from the list of indexed documents, not from policy text".

2.2 Misspelt follow-ups no longer get the greeting reply

- **Problem:** "what kind of jpoint partership was used here" was rewritten correctly by Gemini (*"What kind of joint partnership was used by Allina Health and Aetna Insurance Company?"*), but Gemini also labelled it "not a policy question", so the app skipped the search (0 rounds, 0 chunks) and showed the greeting.
- **Change:** `backend/app/agents/query_planner.py` – only greetings and thanks (detected by the app itself, without Gemini) skip the search. Gemini's label is recorded as a note but no longer decides.
- **Effect:** every real question is searched. An unrelated question simply finds no evidence and the assistant says so.

2.3 Spelling correction before the keyword search

- **Problem:** meaning (vector) search tolerates typos, but keyword search does not: "jpoint" and "partership" match nothing, so only common words matched and unrelated passages (Preventive Care, DME, Strategic Pricer) entered the results.
- **Change:**
    - `keyword_search.unknown_terms()` checks which question words appear in **no** indexed passage (fast, uses the full-text index).
    - If there are any, the Gemini planner runs for that question and corrects the spelling (`prompts.py` now tells it to correct obvious spelling mistakes but keep names, codes and acronyms).
    - Diagnostics show a note, e.g. *Words not found in any policy: jpoint, partership – Gemini corrected the question before searching.*
- **Effect:** typo questions get the same search as correctly spelt ones. Cost: **one extra Gemini planning call** for questions with unknown words (other questions are unchanged). Words that are correct but simply not used in any policy also trigger it; that is harmless.

2.4 Synonyms and acronyms in keyword search

- **Problem:** "abortion" matched an extra section (plan exclusions, S2) by keyword, "termination of pregnancy" did not, so the two answers differed.
- **Change:** new file `backend/app/retrieval/synonyms.py`. Before every keyword search, equivalent terms found in the query are added:
    - **built-in:** abortion ↔ termination of pregnancy; precertification ↔ prior authorization ↔ preauthorization; non-participating ↔ out-of-network; joint venture ↔ joint partnership;
    - **learned from the policies:** every definition like "Coordination Of Benefits (COB)" or "Durable Medical Equipment (DME)" (refreshed automatically when the index changes);
    - **optional:** your own groups in a JSON file (`SYNONYMS_FILE`).
    - Two-letter acronyms only match in capitals ("IT", not "it").
    - Settings: `SYNONYM_EXPANSION=true`, `SYNONYMS_FILE=`.

- **Effect:** different wordings of the same question find the same passages, so answers are more consistent. Vector search and the question itself are unchanged. The first question after starting or re-indexing scans the policies once for acronym definitions (time not measured on the laptop yet).

2.5 Following referenced policies

- **Problem:** the abortion passage says *refer to the "Precertification – Policy – Precertification: Nonparticipating Provider Authorizations" policy*. That policy was not searched, so the evidence check flagged the answer as incomplete.
- **Change:** `backend/app/agents/retrieval_agent.py` – when the evidence is not yet sufficient, policies named in quotes or as Markdown links after "refer to / see / per / as described in" in the relevant passages are searched in the next round, before Gemini's own follow-up queries. Diagnostics show "Next round searches referenced: …".
- **Effect:** answers that depend on another policy become more complete. It only happens when evidence is insufficient and a round is left (max 3 rounds), so simple questions are not slowed down.

2.6 How relevance and order are decided (diagnostics)

- **How it works (unchanged):** search returns candidates ordered by RRF. Gemini's **evidence check** reads the top candidates and returns which ones are actually relevant (`relevant_source_ids`). Those go **first** in the answer context, then the rest by RRF. This is why a passage with a lower RRF score can be #1. In industry terms this is an **LLM reranker**.
- **Change:** the Diagnostics table is now titled "Context passed to the answer" with the explanation *"Passages the evidence check marked relevant (✓) come first, then the rest by RRF score"*, and a new **Relevant ✓** column. Planner notes (e.g. spelling) are shown.
- **Effect:** no change to answers; the order is now explained instead of looking like a bug.

2.7 Softer note for verified partial answers

- **Change:** `frontend/components/chat/assistant-message.tsx` – when an answer is marked *insufficient evidence* **but** all its citations were verified, a small grey note "Partial answer – the cited passages are verified" is shown instead of the yellow warning. Answers without verified citations still show the yellow warning.
- **Effect:** users are not alarmed by correct, cited answers that cover only part of a broad question.

2.8 More passages per answer and recommended settings

Setting	Before	Now	Effect
`TOP_K_FINAL`	8	12	More passages kept from each search
`MAX_CONTEXT_CHUNKS`	12	16	More passages given to Gemini
`MAX_CONTEXT_CHARS`	24,000	40,000	Room for those passages
`GEMINI_TEMPERATURE`	0.1	**0.1 (unchanged)**	Answers may vary in wording
`GEMINI_THINKING_BUDGET`	0 on the laptop	**1024 recommended**	Gemini reasons before answering; use 512 if "malformed JSON" returns

- **Effect:** broad questions (e.g. COB) get more complete answers. Each answer sends more text to Gemini, so token use and time per answer go up somewhat (not measured yet; check the Tokens line in Diagnostics).

2.9 Weekly update in one click

- **Change:** new `update-windows.bat` (place it next to `scrape_epolicies.py`). It:
  1. starts the database if needed,
  2. checks the Google Cloud login,
  3. backs up `output` → `output_backup` and `embedding.jsonl` → `embedding-backup.jsonl`,
  4. runs `scrape_epolicies.py --update` (asks you to close Edge first),
  5. runs `check-files` and `ingest --dry-run` and **asks Y/N** after showing new / modified / deleted counts,
  6. runs `ingest` (only changed files go to Gemini) and `status`.
- **Effect:** the weekly update is one double-click, with a safety check before anything changes. The backend and website keep running; no restart needed.

2.10 Found by testing 3 real policies (Abortion, Accident, Advance Directives)

The 3 policies were typed in from photos and tested offline on the Mac (real database and search code, fake embeddings, no Gemini – nothing left the Mac).

Finding	Fix	Result on the 3 policies
Parser bug: the "Telemedicine Vendors" step table is a list nested ~20 levels deep. markdown-it stops at 20 levels, so **every later section** (Multifetal, Precertification and Referrals, Procedure Codes, Travel, Spontaneous Abortion) was stored under the "Telemedicine Vendors" heading	Nesting limit raised to 100 (`markdown_parser.py`); `CHUNKER_VERSION` v2	All 16 sections detected with their own headings (before: the last 9 chunks were all "Telemedicine Vendors")
Precertification question ranked the wrong section first	Comes from the parser fix + synonyms	"Precertification and Referrals for Termination of Pregnancy" is **#1** for both "abortion" and "termination of pregnancy"; same top-5 passages for both wordings
Acronyms defined in **Acronym Tables** (`| MSA | Medicare Set Aside |`) were not learned	Acronym tables are read too (`synonyms.py`)	10 acronyms learned: ASH, DOH, E&M, EOC, IFP, MA, MSA, NY, SPU, WC (before: 5)
References written as `**See:** [Policy] (link)` were not followed	Reference pattern accepts "See:", "see -" and lists after "See:"	6 referenced policies found, including *Precertification – Nonparticipating Provider Authorizations*

Effect on the 235 policies: any other policy with a step table nested this deep is affected the same way (not yet checked on the laptop). The next `ingest` (or `update-windows.bat`) **re-chunks every document automatically** because the chunker version changed. Unchanged passages reuse their cached embeddings, so only passages whose text changed are sent to Gemini.

3. Earlier changes in the same period

- **Home page:** example questions removed, so users type their own.
- **Theme button:** fixed the red "hydration" warning in development.

- **`start-windows.bat` / `stop-windows.bat`**: one-click start and stop; finds the Python environment in `backend\.venv`, the project folder or the folder above.

4. What did not change

- The embedding model (`gemini-embedding-001`, 768 numbers per chunk). One `ingest` is needed for the parser fix (2.10); cached embeddings are reused.
- RRF formula and `RRF_K=60`.
- Strict grounding and citation verification.
- No new Python or website libraries – nothing to install.

5. Impact summary

Area	Impact
Answer quality	Better for collection questions, typos, different wordings, cross-referenced policies and broad questions
Consistency	Same passages for different wordings; wording still varies a little (temperature 0.1)
Speed	Collection questions: instant. Typo questions: +1 Gemini call. Answers with more context: somewhat slower (not measured)
Cost	Collection questions: free. Slightly more tokens per normal answer (more passages)
Safety	Unchanged: answers only from cited, verified policy passages
Operations	One-click start, stop and weekly update

6. How to update the laptop

1. In PowerShell, go to the project folder and download the 28 changed files from GitHub (your `.env`, `output` and embeddings are not touched):

```
cd C:\Users\n692121\Downloads\epolicies-rag-main\epolicies-rag-main
$base = "https://raw.githubusercontent.com/charan4590/epolicies-rag/main"
$files = ".env.example","README.md","START-HERE.md","update-
windows.bat","backend/app/agents/catalog.py","backend/app/agents/prompts.py","backend/app/
agents/query_planner.py","backend/app/agents/retrieval_agent.py","backend/app/config.py","
backend/app/database/repository.py","backend/app/ingestion/markdown_parser.py","backend/ap
p/ingestion/chunker.py","backend/app/ingestion/pipeline.py","backend/app/ingestion/jsonl_s
tore.py","backend/tests/unit/test_markdown_ingestion.py","backend/app/retrieval/hybrid_sea
rch.py","backend/app/retrieval/keyword_search.py","backend/app/retrieval/ranking.py","back
end/app/retrieval/synonyms.py","backend/app/services/rag_service.py","backend/tests/integr
ation/test_postgres_pipeline.py","backend/tests/unit/test_catalog.py","backend/tests/unit/
test_jsonl_schemas_service.py","backend/tests/unit/test_retrieval_and_agents.py","backend/
tests/unit/test_synonyms.py","frontend/components/chat/assistant-
message.tsx","frontend/components/chat/diagnostics-panel.tsx","frontend/lib/types.ts"
foreach ($f in $files) { Invoke-WebRequest "$base/$f" -OutFile $f; Write-Host "updated $f"
}
```

2. In `.env`, set: `TOP_K_FINAL=12`, `MAX_CONTEXT_CHUNKS=16`, `MAX_CONTEXT_CHARS=40000`, `SYNONYM_EXPANSION=true`, `GEMINI_THINKING_BUDGET=1024`. Keep `GEMINI_TEMPERATURE=0.1` and `GEMINI_MODEL=gemini-2.5-flash`.
3. Re-index once so the parser fix applies to all 235 policies (only changed passages are embedded):

```
cd backend
python -m app.cli ingest
```

4. Double-click `stop-windows.bat`, then `start-windows.bat`.
5. Test: "How many policies are there?", a misspelt follow-up, the abortion / termination-of-pregnancy precertification pair, and a COB question with Diagnostics on.

7. How to measure the improvement (next step)

The project's evaluation tool (`python -m app.cli evaluate --with-answers --judge`) reports **Recall@K**, **MRR**, **citation precision / recall**, **faithfulness (proxy)**, **insufficient-evidence accuracy** and an automated **answer relevance** score (1–5). It needs a test set of 20–30 real questions with the correct policy file for each (`backend/eval/epolicies_eval.jsonl`, from the template). Running it before and after these changes would give real numbers to show the improvement.